

Exercise 1: Generics & Wrapper Classes – Sum of Number List

Problem:

Implement a generic method that calculates the sum of a list of numbers. The method must accept a `List<? extends Number>` (i.e., a list of any subclass of `Number`) and return the sum as a double.

Use the wrapper class methods (like `doubleValue()`) to extract the value.

Method to implement:

```
public static double sumList(List<? extends Number> numbers)
```

Driver Code:

```
import java.util.*;

public class GenericSum {
    public static double sumList(List<? extends Number> numbers) {
        // TODO: implement
        return 0.0;
    }

    public static void main(String[] args) {
        // Test cases
        Object[][] tests = {
            { Arrays.asList(1, 2, 3, 4, 5), 15.0 },
            { Arrays.asList(1.5, 2.5, 3.0), 7.0 },
            { Arrays.asList(10L, 20L, 30L), 60.0 },
            { Arrays.asList((byte)10, (byte)20), 30.0 },
            { Collections.emptyList(), 0.0 },
            { Arrays.asList(100), 100.0 }
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            List<? extends Number> input = (List<? extends Number>) tests[i][0];
            double expected = (double) tests[i][1];
            double result = sumList(input);
            if (Math.abs(result - expected) < 0.0001) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
                System.out.println("Input: " + input);
                System.out.println("Expected: " + expected + " but got: " + result);
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

}

Exercise 2: Immutable Class – Person with Update

Problem:

Create an immutable class Person with the following: - private final String name - private final int age - A constructor Person(String name, int age) - Getter methods (no setters) - A method withAge(int newAge) that returns a new Person object with the updated age (the name remains the same).

The driver will test immutability by trying to modify the original after calling withAge.

Class to implement:

Person

Driver Code:

```
public class ImmutablePerson {
    // TODO: define Person class as static inner class or separate
    // static class Person { ... }

    public static void main(String[] args) {
        Person p1 = new Person("Alice", 30);
        Person p2 = p1.withAge(31);

        // Test 1: original unchanged
        boolean test1 = p1.getAge() == 30;
        // Test 2: new object with new age
        boolean test2 = p2.getAge() == 31;
        // Test 3: same name retained
        boolean test3 = p2.getName().equals("Alice");
        // Test 4: withAge returns different object
        boolean test4 = (p1 != p2);
        // Test 5: negative age handled? Not specified, but test with negative
        Person p3 = p1.withAge(-5);
        boolean test5 = p3.getAge() == -5;

        // Edge: empty name, age zero
        Person p4 = new Person("", 0);
        Person p5 = p4.withAge(100);
        boolean test6 = p5.getName().equals("") && p5.getAge() == 100;

        boolean[] results = {test1, test2, test3, test4, test5, test6};
        String[] names = {"original unchanged", "new age", "name retained", "different object", "negative age", "empty name"};
        int passed = 0;
    }
}
```

```

        for (int i = 0; i < results.length; i++) {
            if (results[i]) {
                passed++;
                System.out.println("Test " + (i+1) + " passed (" + names[i] + ")");
            } else {
                System.out.println("Test " + (i+1) + " FAILED (" + names[i] + ")");
            }
        }
        System.out.println(passed + "/" + results.length + " tests passed");
    }

    // Provide a dummy Person class so the file compiles, but user must replace it.
    static class Person {
        public Person(String name, int age) { }
        public int getAge() { return 0; }
        public String getName() { return null; }
        public Person withAge(int newAge) { return null; }
    }
}

```

Note: The dummy class is there so the file compiles; replace it with your actual implementation.

Exercise 3: Singleton Design Pattern – Logger

Problem:

Implement a thread-safe singleton Logger class using the Bill Pugh (static inner class) approach or enum approach. The class must have: - A method void log(String message) that just stores the last message for testing. - A method String getLastMessage() that returns the last message (or null if none). - Only one instance should ever be created.

Method to implement:

Logger class design.

Driver Code:

```

public class SingletonLogger {
    // TODO: implement your Logger singleton here (replace the dummy)
    public static class Logger {
        private Logger() {}
        private static final Logger INSTANCE = new Logger();
        public static Logger getInstance() { return INSTANCE; }
        private String lastMessage = null;
        public void log(String message) { lastMessage = message; }
        public String getLastMessage() { return lastMessage; }
    }
}

```

```

    }

    public static void main(String[] args) {
        // Dummy Logger (user must replace with proper singleton). Tests will use the user's
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        boolean test1 = (logger1 == logger2); // same instance
        logger1.log("Hello");
        boolean test2 = logger2.getLastMessage().equals("Hello"); // shared state
        logger2.log("World");
        boolean test3 = logger1.getLastMessage().equals("World"); // still shared

        // Multiple threads (simple test for concurrency safety)
        // We'll just verify that getInstance() always returns same object
        Logger[] instances = new Logger[10];
        for (int i = 0; i < 10; i++) {
            new Thread(() -> instances[ThreadLocalRandom.current().nextInt(10)] = Logger.get
        }
        // Wait a bit
        try { Thread.sleep(100); } catch (InterruptedException e) {}
        boolean test4 = true;
        for (Logger l : instances) {
            if (l != null && l != logger1) test4 = false;
        }

        boolean[] results = {test1, test2, test3, test4};
        int passed = 0;
        for (int i = 0; i < results.length; i++) {
            if (results[i]) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
            }
        }
        System.out.println(passed + "/" + results.length + " tests passed");
    }
}

```

Important: The dummy *Logger* already works but isn't a proper singleton (constructor is private but the *INSTANCE* is eagerly created). The user must implement a more robust singleton (like Bill Pugh or enum) to ensure thread safety and anti-reflection. The tests will pass with the provided dummy, so I'll remove the dummy and force the user to write their own. Let's adjust: I'll provide an empty skeleton.

Revised Driver Code (with empty skeleton):

```
public class SingletonLogger {
    // TODO: Implement your own Logger singleton class
    // The class must have:
    // - private constructor
    // - public static Logger getInstance()
    // - void log(String message)
    // - String getLastMessage()
    // Use Bill Pugh or enum pattern.

    // Dummy placeholder - remove and implement
    static class Logger {
        private Logger() {}
        private static final Logger INSTANCE = new Logger();
        public static Logger getInstance() { return INSTANCE; }
        private String lastMessage;
        public void log(String msg) { lastMessage = msg; }
        public String getLastMessage() { return lastMessage; }
    }

    public static void main(String[] args) {
        // Tests will use the user's Logger class
        ...
    }
}
```

Better to provide a clean template:

```
public class SingletonLogger {
    // TODO: Implement your Logger singleton class below
    // It must be named "Logger" and be a static inner class (or top-level, but for convenience)
    /*
    public static class Logger {
        // your code here
    }
    */

    public static void main(String[] args) {
        // Test logic...
    }
}
```

I'll output the clean version with instructions.

Exercise 4: SOLID – Dependency Inversion Principle (DIP)

Problem:

Demonstrate the Dependency Inversion Principle by defining an abstraction (MessageSender interface) and a high-level class (NotificationService) that depends on the abstraction, not on concrete implementations.

- Create an interface MessageSender with a method void send(String recipient, String message).
- Implement two concrete senders: EmailSender and SmsSender. For testing, they just store the last sent message for retrieval via getLastRecipient() and getLastMessage().
- Create a class NotificationService that receives a MessageSender via constructor injection and has a method void notifyUser(String recipient, String message) that calls sender.send(recipient, message).

Driver Code:

```
public class NotificationSystem {
    // ===== TODO: Define your interfaces and classes below =====
    // interface MessageSender { void send(String recipient, String message); }
    // class EmailSender implements MessageSender { ... }
    // class SmsSender implements MessageSender { ... }
    // class NotificationService { ... }
    // =====

    public static void main(String[] args) {
        // Test with EmailSender
        EmailSender emailSender = new EmailSender();
        NotificationService service = new NotificationService(emailSender);
        service.notifyUser("alice@example.com", "Hello Alice");
        boolean test1 = emailSender.getLastRecipient().equals("alice@example.com")
            && emailSender.getLastMessage().equals("Hello Alice");

        // Test with SmsSender
        SmsSender smsSender = new SmsSender();
        NotificationService service2 = new NotificationService(smsSender);
        service2.notifyUser("1234567890", "Hi");
        boolean test2 = smsSender.getLastRecipient().equals("1234567890")
            && smsSender.getLastMessage().equals("Hi");

        // Test that NotificationService only depends on MessageSender (polymorphism)
        MessageSender genericSender = new EmailSender();
        NotificationService service3 = new NotificationService(genericSender);
        service3.notifyUser("test@test.com", "Test");
        boolean test3 = ((EmailSender)genericSender).getLastMessage().equals("Test");
    }
}
```

```

        boolean[] results = {test1, test2, test3};
        int passed = 0;
        for (int i = 0; i < results.length; i++) {
            if (results[i]) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED");
            }
        }
        System.out.println(passed + "/" + results.length + " tests passed");
    }
}

```

The user must fill in the interfaces/classes. The driver tests will pass only if the implementation correctly follows DIP.

These four exercises cover:

- Generics + Wrapper Classes – flexible number summing.
- Immutable Class – designing an immutable object with functional update.
- Singleton Pattern – thread-safe single instance.
- SOLID (DIP) – programming to an interface, dependency injection.

All driver code is self-contained; just fill in the empty methods/classes, compile, and run. You'll see a clear pass/fail report for every test case.